Seminarium dyplomowe

Układ aktywnego tłumienia hałasu pod nadzorem jądra systemu Linux z rozszerzeniami real-time

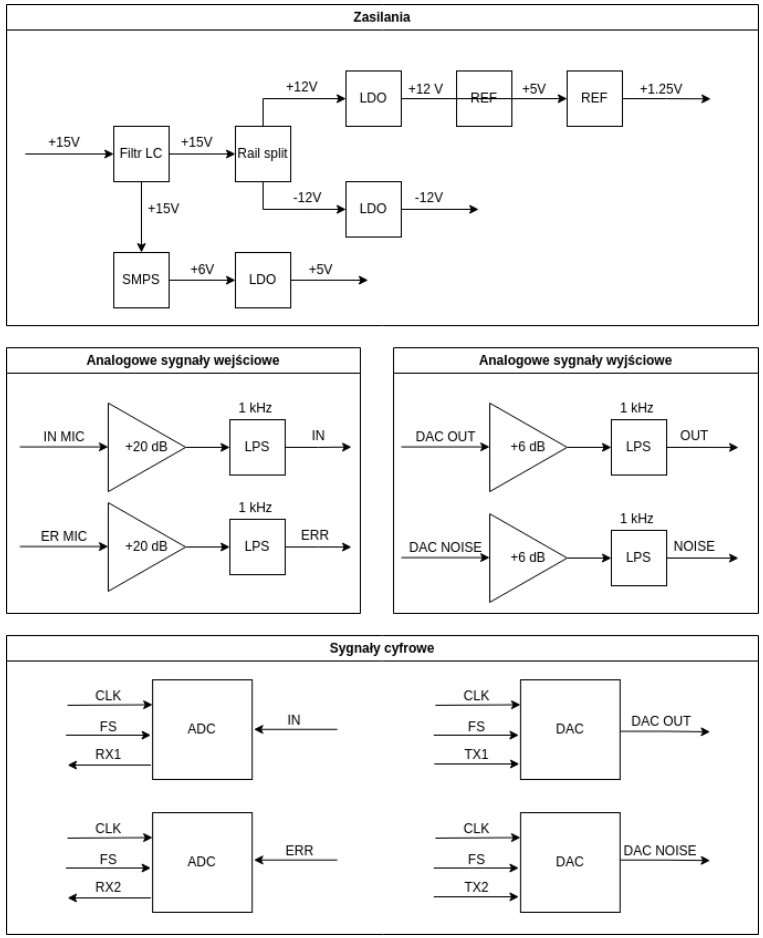
Daniel Kopiecki

Politechnika Śląska

21 marca 2023

Część sprzętowa

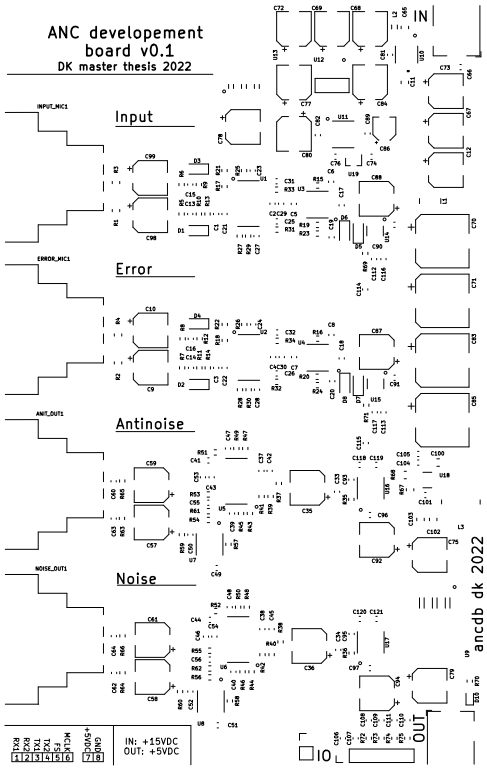
Uproszczony schemat architektury

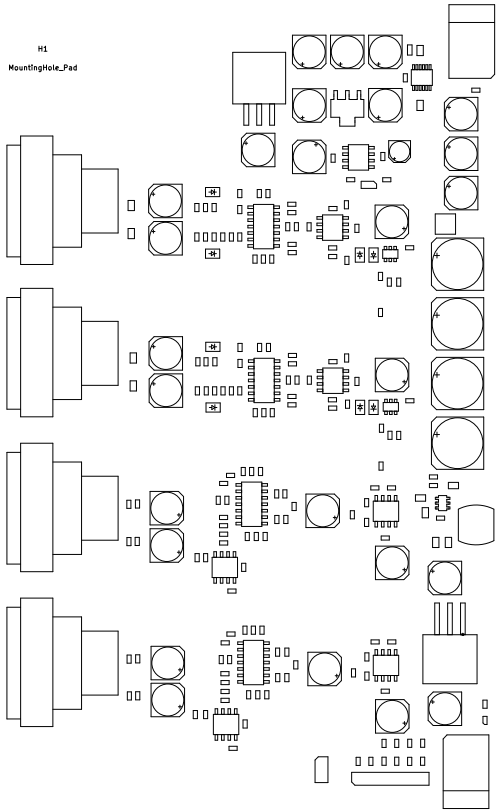


Rysunek: Architektura sprzętowa

Część sprzętowa

Warstwa przednia

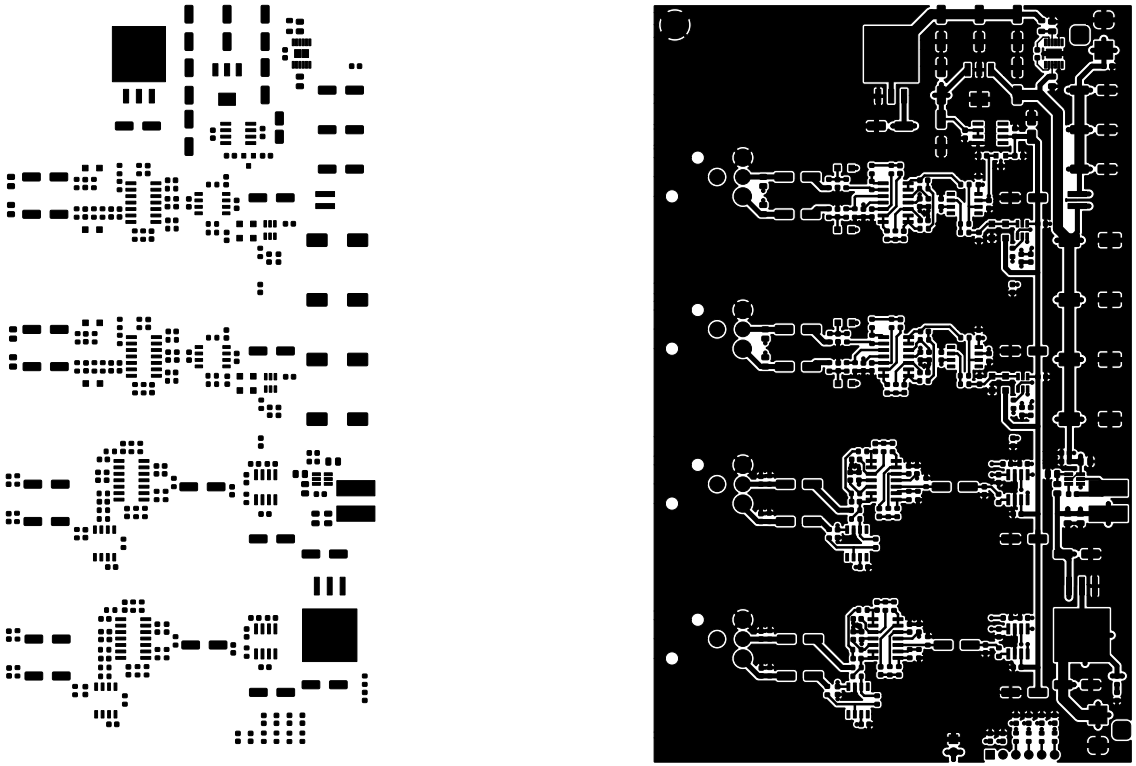




Rysunek: Warstwa przednia, 1

Część sprzętowa

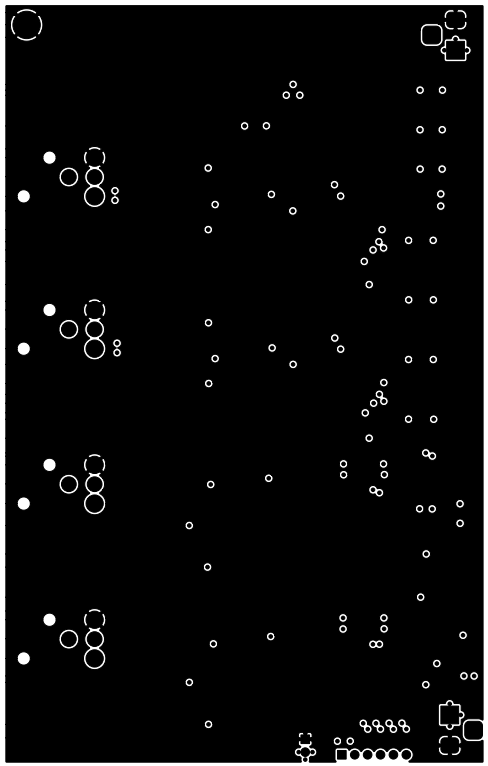
Warstwa przednia

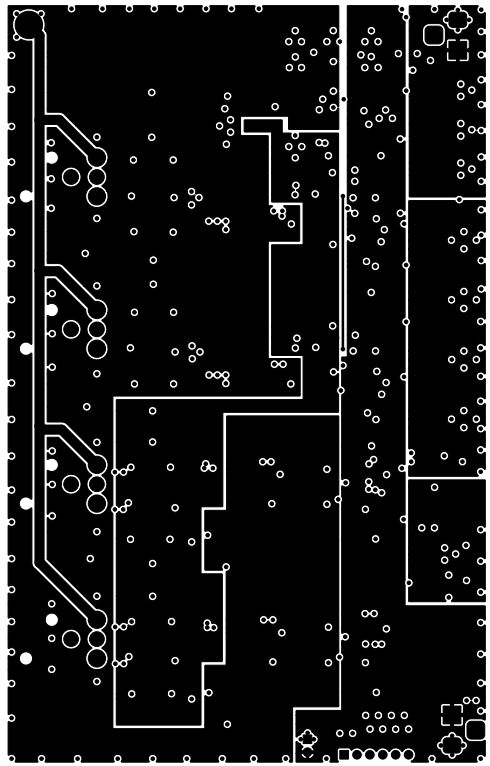


Rysunek: Warstwa przednia, 2

Część sprzętowa

Warstwy środkowe

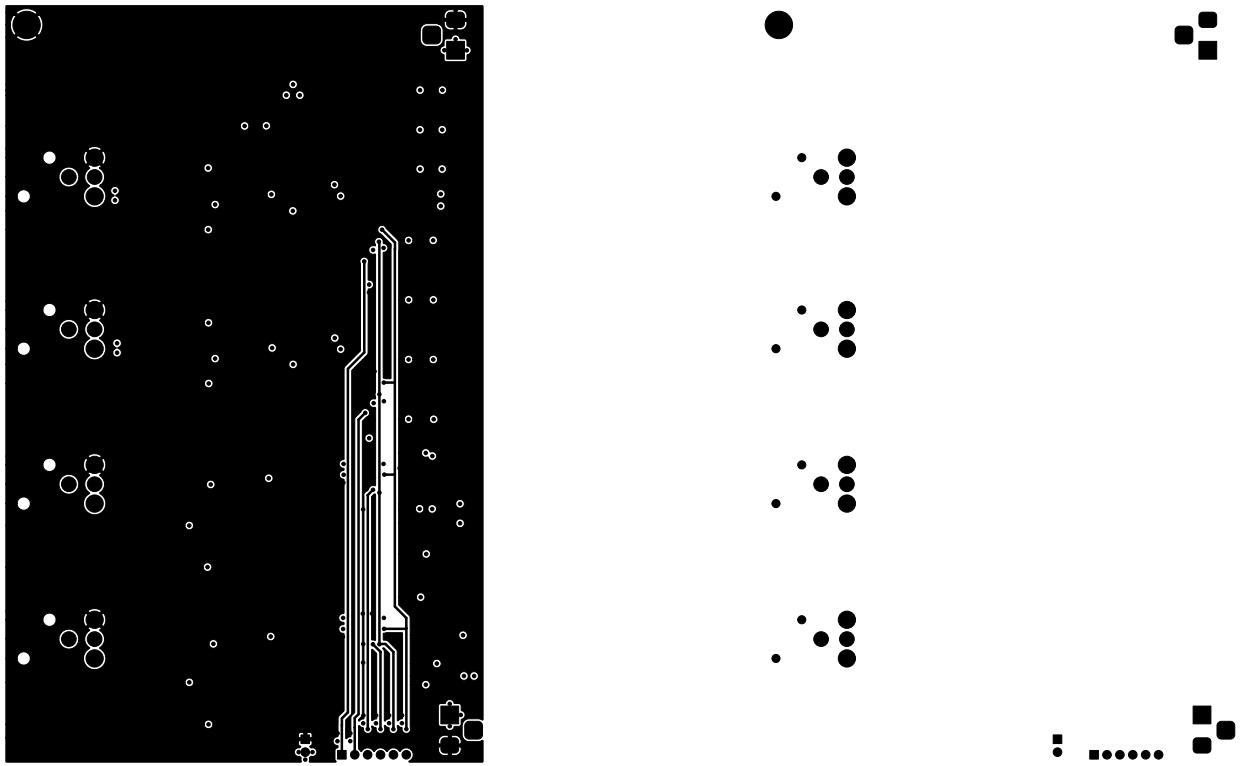




Rysunek: Warstwy środkowe

Część sprzętowa

Warstwa dolna

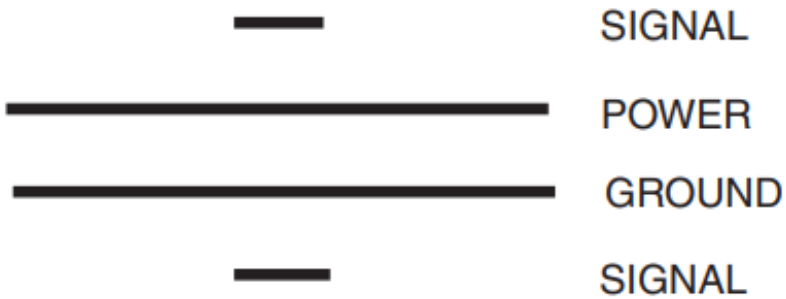


Rysunek: Warstwa dolna

Część sprzętowa

Kilka ciekawych kwestii implementacyjnych

- Wszystko wpływa na wszystko
- Odległość pomiędzy ścieżkami ma **kolosalne znaczenie**
- Kwestie obligatoryjne - rozłożenie warst (*stackup*) oraz umiejscowienie komponentów



Część sprzętowa

Kilka ciekawych kwestii implementacyjnych

- Płaszczyzną odniesienia dla relatywnie szybkich sygnałów cyfrowych może w równym stopniu być płaszczyzna masy, jak i płaszczyzna **zasilania**
- Z Henry W. Ott, *Electromagnetic Compatibility Engineering* - układ scalony **nie jest źródłem prądu** sam w sobie, lecz stanowi jedynie jego **sterownik**. Prawdziwym źródłem jest kondensator odsprzęgający lub/i pasożytnicza pojemność pomiędzy ścieżkami/płaszczyznami. endframe

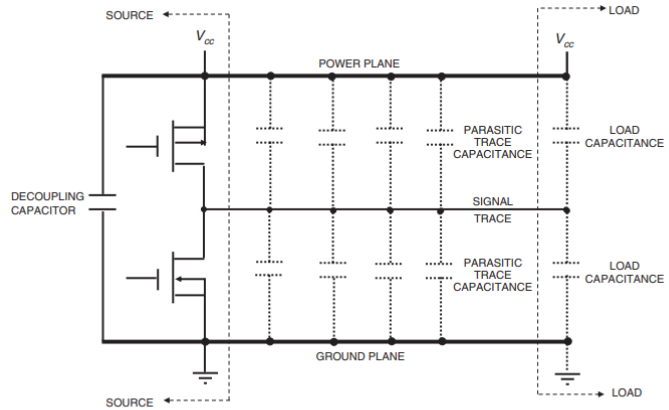


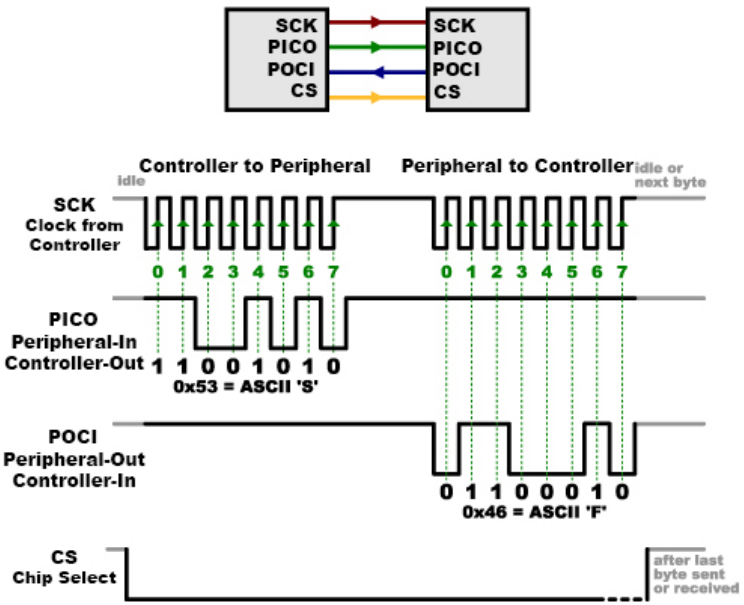
FIGURE 10-31. Circuit of a CMOS logic gate driving a stripline located between a power and ground plane.

Rysunek: Możliwe drogi sygnałów cyfrowych

Część sprzętowa

Wykorzystywany protokół cyfrowy

- Bardzo ważną kwestią implementacyjną jest opracowanie synchronicznego próbkowania wszystkich przetworników. Aby to uzyskać, konieczny jest protokół cyfrowy, który zapewni zegar, odpowiednią manipulację selektorem, dwie linie RX oraz dwie linie TX.

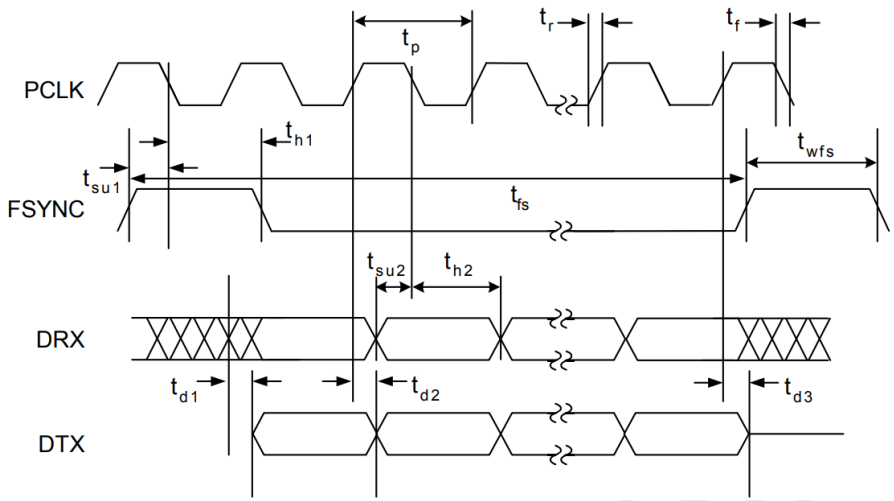


Rysunek: Protokół SPI - *Serial Peripheral Interface*

Część sprzętowa

Protokół cyfrowy - możliwe rozwiązania

- McASP - protokół cyfrowy opracowany przez Texas Instruments w celu interakcji z wielokanałowymi kartami dźwiękowymi - podejście **optymalne**



Rysunek: Protokół McASP - *Multichannel Audio Serial Port*

Część sprzętowa

Protokół cyfrowy - możliwe rozwiązania

- *Bit banging* - metoda transmisji danych oparta na implementacji dedykowanego protokołu cyfrowego z wykorzystaniem dwustanowych pinów wejściowo-wyjściowych (*GPIOs*) - podejście **ostateczne**. Wymaga implementacji sterownika na najniższym poziomie.

Część sprzętowo-programowa

Protokół cyfrowy - bit banging, przykład

```
void tx_byte(uint8_t data)
{
        gpio_low(SD_CS);
        for (uint8_t i = 0u; i < 8u; ++i) {
                if (data & 0x80u)
                        gpio_high(SD_DI);
                else
                        gpio_low(SD_DI);

                gpio_low(SD_CLK);
                delay();
                gpio_high(SD_CLK);

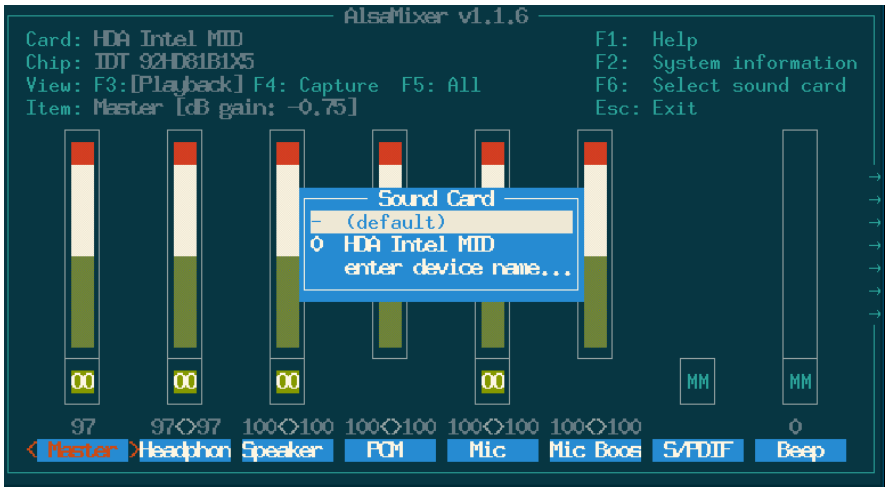
                data <<= 1u;
        }

        gpio_low(SD_CS);
}
```

Część programowa

ALSA

- Na chwilę obecną wybrana została ścieżka **optymalna**.
- Tak jak pisano wcześniej, protokół McASP wykorzystywany jest do interakcji z kartami dźwiękowymi.
- Głównym środowiskiem do interakcji z urządzeniami audio pod systemem operacyjnym GNU/Linux jest **ALSA** (*Advanced Linux Sound Architecture*)

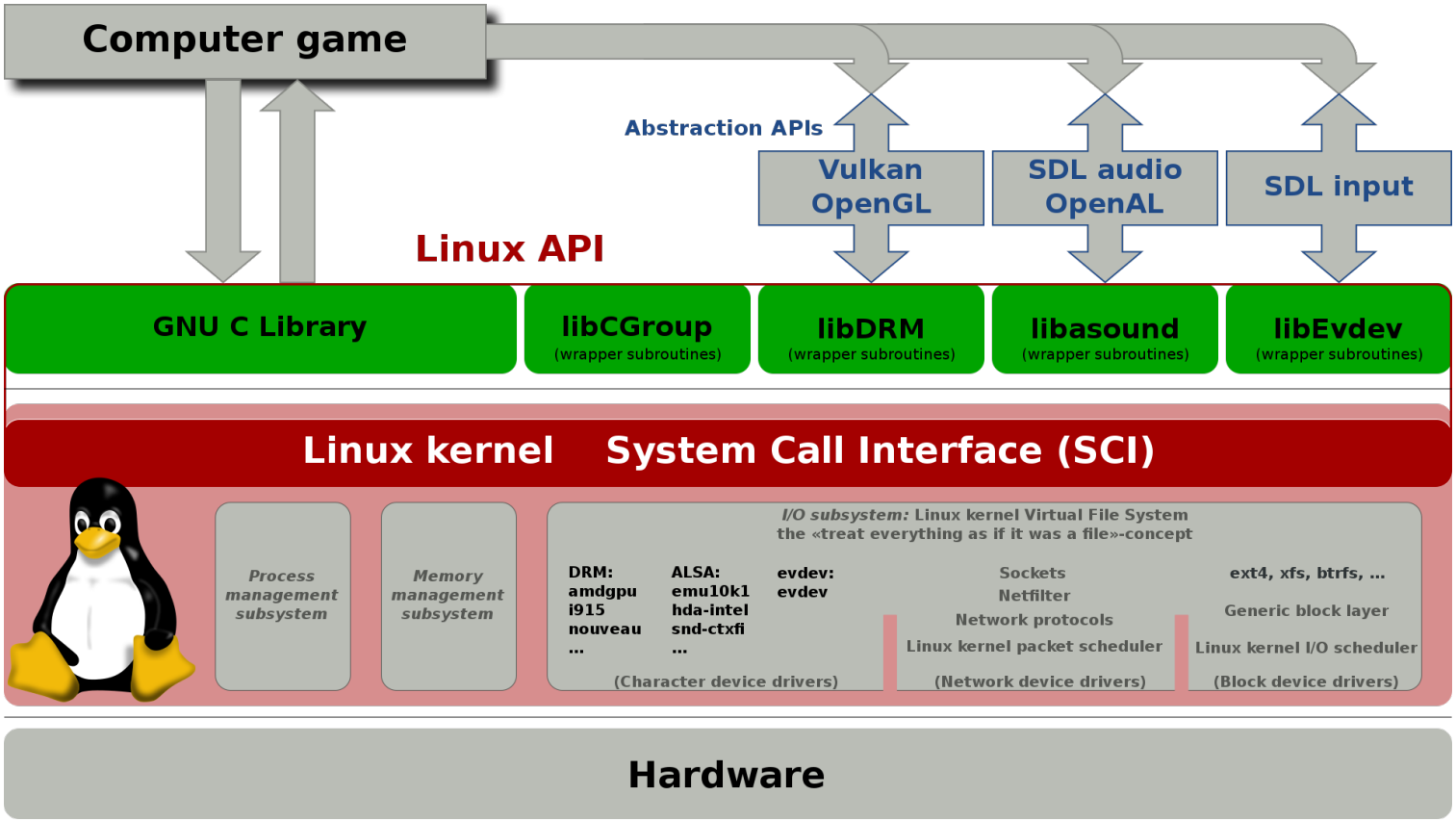


Rysunek: ALSA - widok aplikacji *AlsaMixer* ($ alsamixer)

Część programowa

ALSA

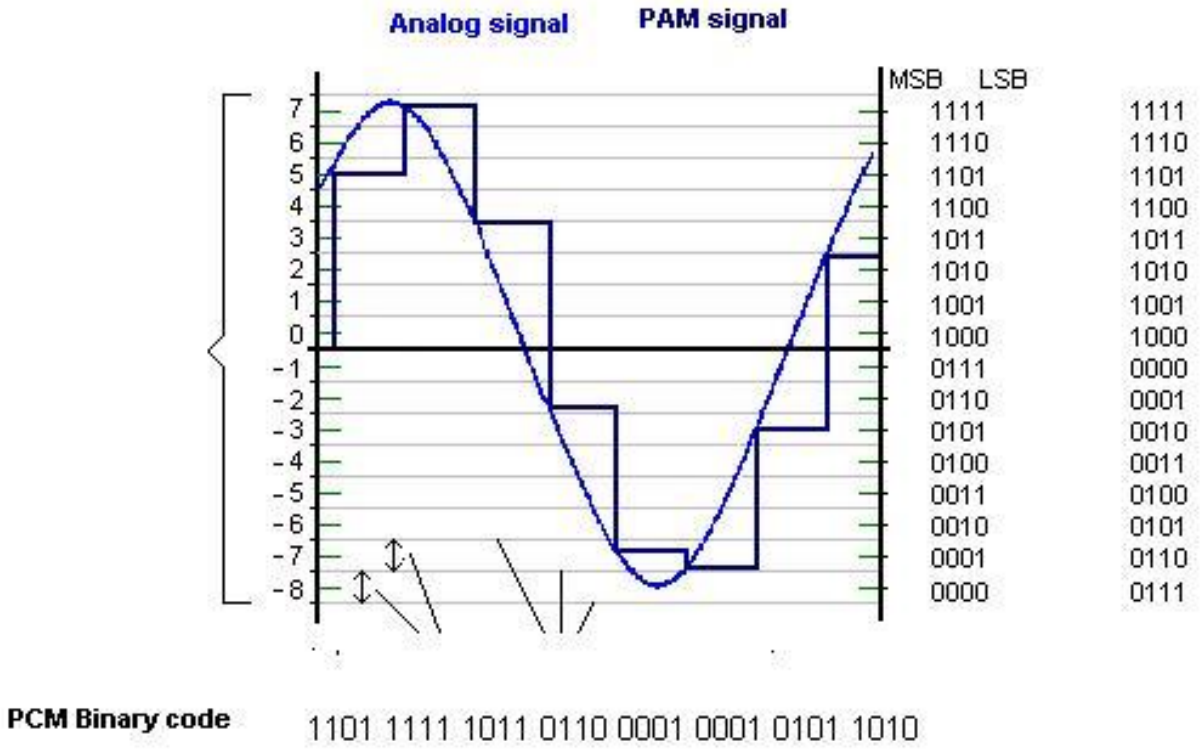
- **ALSA** udostępnia duże, niskopoziomowe API do interakcji z kartami dźwiękowymi z poziomu przestrzeni użytkownika (*userspace*).
- Plik nagłówkowy biblioteki nosi nazywę *asoundlib.h* a sama biblioteka *libasound*



Część programowa

Cyfrowa reprezentacja dźwięku

- Cyfrową reprezentacją dźwięku, wykorzystywaną przez środowisko **ALSA** jest modulacja pulsowo-kodowa (*PCM - Pulse Code Modulation*)

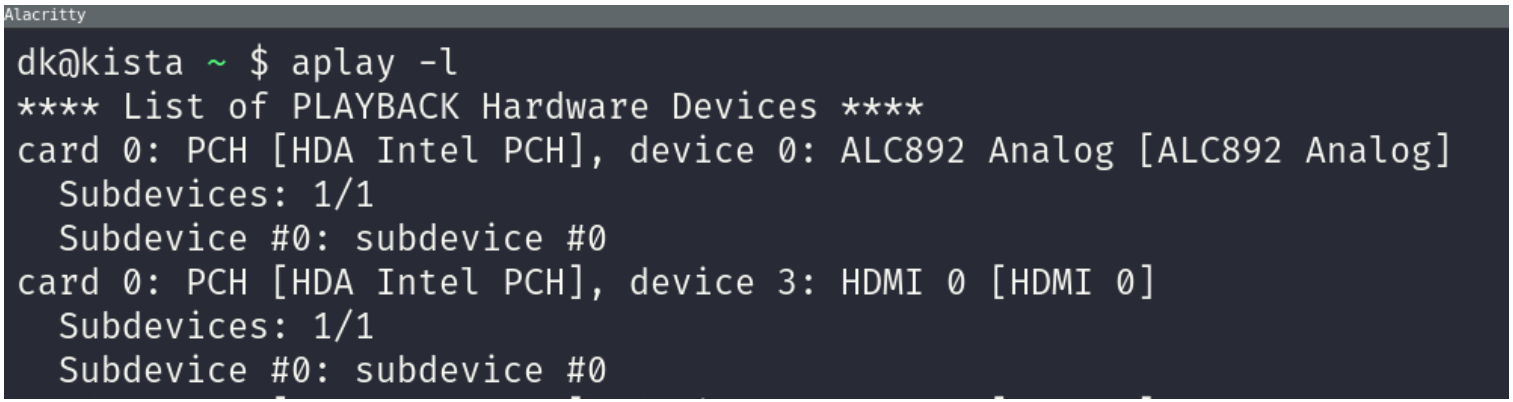


Rysunek: Modulacja pulsowo-kodowa

Część programowa

ALSA

- ALSA korzysta ze specjalnego nazewnictwa urządzeń dźwiękowych dostępnych dla użytkownika. Mogą to być rzeczywiste urządzenia (karty dźwiękowe), lub ich wirtualne reprezentacje.
- W przypadku fizycznych urządzeń, korzysta się z formatu *hw:i,j*, gdzie *i* oznacza numer karty, a *j* numer urządzenia kontrolowanego przez kartę, czyli przypkładowo zestawu głośników lub mikrofonu.
- W przpyadku wirtualnych urządzeń, korzysta się z formatu *plughw:i, j*.

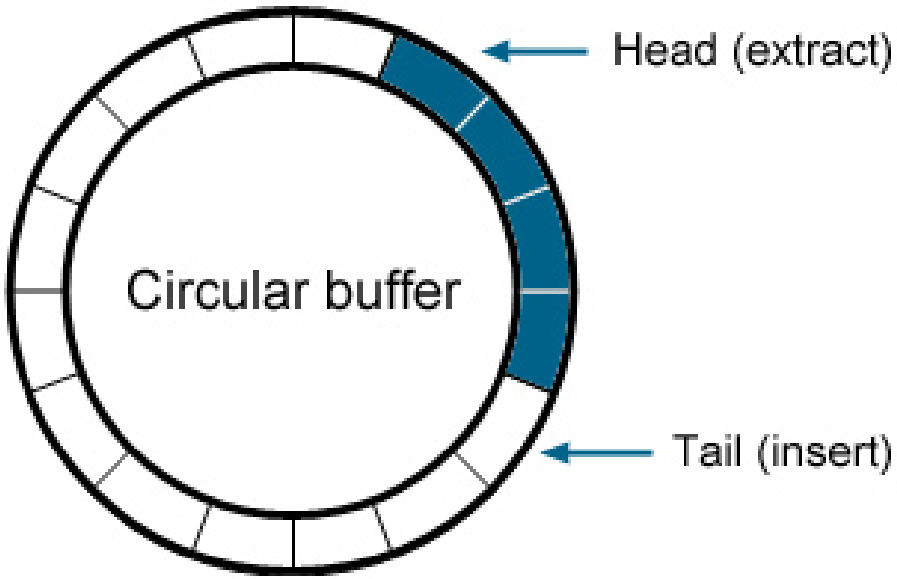


Rysunek: Rezultat polecenia *aplay -l*

Część programowa

ALSA - lokowanie próbek

- Na poziomie sterownika jądra sprzętu audio, **ALSA** tworzy bufor cykliczny, którego długość można modyfikować używając udostępnionego API.
- Jeżeli bufor jest zapełniony, sterownik jądra generuje przerwanie. Rolę obsługi przerwania obejmuje DMA (*Direct Memory Access*), które nieblokująco przenosi próbki z bufora sterownika do bufora aplikacji.

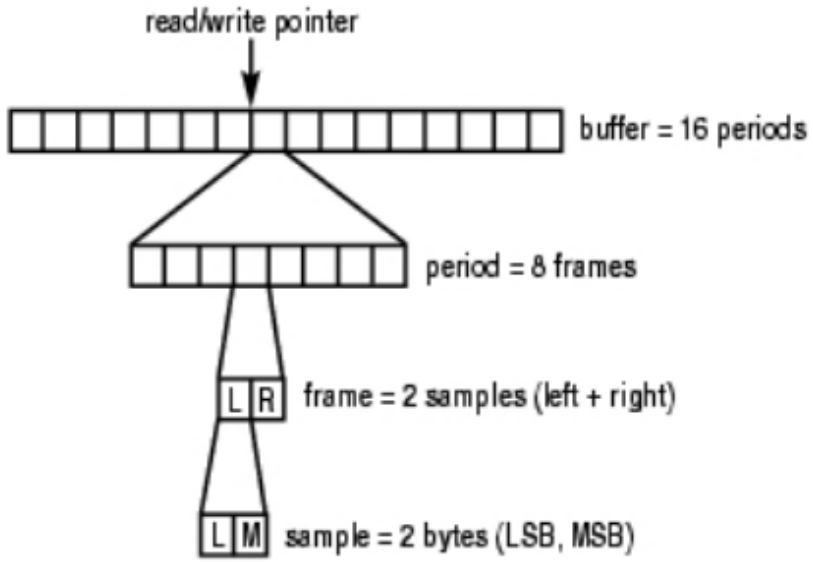


Rysunek: Bufor cykliczny

Część programowa

ALSA - lokowanie próbek

- Bufor tworzony przez sterownik jądra zapełniany jest przez **fragmenty** (lub okresy, *periods*).
- **Fragmenty** składają się z określonej liczby **ramek** (*frames*).
- **Ramka** składa się z próbek dla każdego kanału karty dźwiękowej (liczba kanałów może być wyspecifikowana, tak jak format próbki).



Rysunek: Podział danych

ALSA - bufory

- Jeżeli aplikacja jest uruchomiona, to dane w nieprzerwany sposób są transportowane z/do bufora cyklicznego sterownika jądra do/z bufora aplikacji.
- W przypadku odczytywania danych (*capture*), jeżeli nie są one odpowiednio szybko czytane, to może dojść do sytuacji, kiedy dane w buforze cyklicznym zostaną nadpisane nowymi. Taka sytuacja nosi nazwę **buffer overrun**.
- W przypadku zapisywania danych (*playback*), jeżeli nie są one odpowiednio szybko wysyłane, to może dojść do odwrotnej sytuacji, czyli bufor cykliczny w pewnym momencie się całkowicie opróżni. Taka sytuacja nosi nazwę **buffer underrun**.

Część programowa

ALSA - latencja

- Bardzo ważnym pojęciem związanym z przetwarzaniem sygnału audio jest **latencja**
- **Latencja** to nic innego jak opóźnienie, które towarzyszy sygnałowi audio od momentu trafienia na wejście systemu do jego opuszczenia.
- W środowisku **ALSA** latencja definiowana jest następującą zależnością:

$$T_L = \frac{n_f F_{rb}}{f_s P_{rb}} \tag{1}$$

gdzie n_f - ilość fragmentów, F_{rb} - rozmiar fragmentu w bajtach, f_s - częstotliwość próbkowania, P_{rb} - rozmiar próbki w bajtach.

- Licznik równania to nic innego jak zadeklarowany rozmiar bufora w bajtach, czyli całe wyrażenie odpowiada czasowi potrzebnemu na przesłanie całego bufora (zbuforowaniu danych).

Część programowa

ALSA - latencja

Przykładowo

- Chcąc odtworzyć dźwięk z szesnatostobitowego systemu stereo (dwa kanały, głośnik lewy i prawy, próbki po 16 bitów = 2 bajty) z typową częstotliwością audio równą 44100 kHz, nasza **ramka** ma rozmiar 4 bajtów (dwie dwu-bajtowe próbki, jedna na kanał). Deklarując rozmiar **fragmentu** na 2048 **ramek**, otrzymujemy długość **fragmentu** w bajtach równą 8192 bajtów.
- Deklarując rozmiar bufora na dwa fragmenty, otrzymujemy rozmiar bufora równy 16384 bajtów.
- Zbuforowanie tych danych, definiujące latencję, w tak skonstruowanym systemie zajmie $\frac{16384}{44100*4} \approx 0.093$ s.

Część programowa

ALSA - przykładowy prosty program

```
int playback(uint8_t* const buffer, size_t size)
{
        const uint8_t channels          = 1;
        const uint32_t rate             = 48000;
        const char* pcm_name            = "hw:0,0";
        const snd_pcm_stream_t stream   = SND_PCM_STREAM_PLAYBACK;
        const snd_pcm_format_t format   = SND_PCM_FORMAT_U8;
        const snd_pcm_access_t access   = SND_PCM_ACCESS_RW_INTERLEAVED;
        const uint32_t latency_us       = 500000u;

        snd_pcm_t* pcm_handle = NULL;
        int error             = snd_pcm_open(&pcm_handle, pcm_name, stream, 0);
        if (error < 0)
                return error;

        error = snd_pcm_set_params(
                pcm_handle, format, access, channels, rate, true, latency_us);
        if (error < 0)
                return error;

        for (uint8_t i = 0u; i < 16u; ++i) {
                snd_pcm_sframes_t transfered_frames =
                        snd_pcm_writei(pcm_handle, buffer, size);
                if (transfered_frames < 0)
                        xrun_recover(pcm_handle, transfered_frames, 0);
                else if ((transfered_frames > 0) && (transfered_frames < size))
                        LOG_ERROR("Short write!");
        }

        snd_pcm_drain(pcm_handle);
        snd_pcm_close(pcm_handle);
}
```